# Minimizing Maximum Betweenness Centrality via Single Edge Addition: A Comparative Study of Brute Force, Heuristic, ML, and Parallel Approaches

Research Paper

April 19, 2026


## Abstract

We study the problem of reducing the maximum betweenness centrality (BC) node in a graph by adding exactly one edge. This problem is critical for improving network resilience, reducing bottlenecks, and balancing traffic in communication, social, and transportation networks. We compare four approaches: (1) an **exhaustive brute-force** search over all non-edges, (2) a **hop-based heuristic** that restricts the search to the 1-hop and 2-hop neighborhood of the target node, (3) a **probability-enhanced** algorithm using structural feature scoring with early stopping, and (4) a **machine-learning model** (Gradient Boosted Trees) trained on brute-force ground truth. We further explore **parallelization** of the brute-force method using multiprocessing. Our experiments span **85 graphs** across **9 topologies** (Erdős-Rényi, Barabási-Albert, Watts-Strogatz, Path, Barbell, Star, Grid, Random Tree, Powerlaw Cluster, Connected Caveman). The hop-based heuristic achieves mean speedup of 14.28249390795684× with 71.20931067188336% optimality, while the probability-enhanced variant improves candidate ranking, and the ML model learns to predict effective edges from structural features.


## 1 Introduction

Betweenness centrality (BC) is one of the most important metrics in network analysis, measuring how often a node lies on shortest paths between other nodes [1]. Nodes with high BC act as bottlenecks and single points of failure. Reducing the BC of the most central node by strategically adding a single edge has applications in:

- **Network resilience**: Removing bottleneck dependencies.
- **Traffic engineering**: Balancing load across network paths.
- **Social networks**: Reducing information gatekeeping.
- **Infrastructure**: Improving robustness of communication networks.

The naive brute-force approach evaluates every possible non-edge, recomputing BC each time, yielding $O(n^2 \cdot nm)$ complexity. We propose and evaluate multiple faster alternatives that trade marginal optimality for significant computational savings.

### 1.1 Problem Formulation

Given an undirected connected graph $G = (V, E)$, let $v^* = \arg\max_{v \in V} BC(v)$ be the node with maximum betweenness centrality. We seek a non-edge $(u, w) \notin E$ such that when $G' = (V, E \cup \{(u, w)\})$:

$$(u^*, w^*) = \arg \max_{(u,w) \notin E} [BC_G(v^*) - BC_{G'}(v^*)] \tag{1}$$

subject to: $\max_{v \neq v^*}[BC_{G'}(v) - BC_G(v)] \leq \tau \cdot \overline{BC}$, where $\tau$ is a load-balance threshold preventing excessive BC increase on other nodes.

## 2 Related Work

Brandes [2] introduced the $O(nm)$ algorithm for computing betweenness centrality. Several works have studied BC modification through edge operations [3, 4]. Our work uniquely combines heuristic search, probabilistic scoring, machine learning, and parallelization for this specific optimization problem.

## 3 Algorithms

### 3.1 Brute Force (Baseline)

The exhaustive approach enumerates all $|E^c| = \binom{n}{2} - m$ non-edges. For each candidate $(u, w)$, we add it to $G$, recompute BC using Brandes' algorithm, and record the reduction in $BC(v^*)$.

**Algorithm 1** Brute Force BC Minimization

**Require:** Graph $G = (V, E)$, target $v^*$, threshold $\tau$

```
BC_0 ← Brandes(G)
for each non-edge (u, w) ∉ E do
    G' ← G ∪ {(u, w)}
    BC' ← Brandes(G')
    Δ ← BC_0(v*) − BC'(v*)
    if Δ > Δ_best and load constraint satisfied then
        Update best edge
    end if
end for
```

**Complexity:** $O\left(\binom{n}{2} \cdot nm\right) = O(n^3 m)$

### 3.2 Hop-Based Heuristic (Smart Algorithm)

**Key Insight:** Adding an edge far from $v^*$ is unlikely to affect its BC significantly. We restrict candidates to non-edges between nodes in the *1-hop* and *2-hop* neighborhood of $v^*$.

**Candidate Classes:**

1. **Class C1 (2-hop × 2-hop):** Score = 2.0 — Creates strongest bypass around $v^*$.
2. **Class C2 (1-hop × 1-hop):** Score = 1.0 — Triangle closing, reduces dependency.
3. **Class C3 (Mixed):** Score = 0.6 — Partial bypass.

Candidates are sorted by score and capped at $K$ (typically 50). **Complexity:** $O(K \cdot nm)$, where $K \ll \binom{n}{2}$.

### 3.3 Probability-Enhanced Algorithm

We extend the hop-based approach with a composite scoring function:

$$S(u, w) = \alpha_1 D + \alpha_2 P + \alpha_3 B + \alpha_4 J + \alpha_5 \Gamma + \alpha_6 C \tag{2}$$

where:

- $D$: Distance-based prior (hop class weight)
- $P$: Path shortcut value ($\propto$ shortest path length between $u$, $w$)
- $B$: BC contribution estimate
- $J$: Neighborhood diversity ($1 - \text{Jaccard}(N(u), N(w))$)
- $\Gamma$: Degree-based Gaussian prior
- $C$: Endpoint betweenness score

**Early Stopping:** If the best candidate hasn't improved for $K_{\text{stop}}$ consecutive evaluations, we terminate.

### 3.4 ML-Guided Algorithm

We train a Gradient Boosted Trees (GBT) regression model to predict $\Delta BC$ from 26 structural features per candidate edge:

Table 1: Feature categories for ML model

| Category | Features |
|---|---|
| Degree | $\deg(u)$, $\deg(w)$, product, sum |
| Distance | $d(u, v^*)$, $d(w, v^*)$, sum, max |
| Betweenness | $BC(u)$, $BC(w)$, sum, max |
| Neighborhood | Common neighbors, Jaccard, Adamic-Adar |
| Clustering | $C(u)$, $C(w)$, average |
| Path | Shortest path $u \to w$ |
| Graph-level | Density, avg clustering, $n$, $m$ |
| Target | $BC(v^*)$, $\deg(v^*)$ |

**Training:** Ground truth from brute-force evaluation on training graphs. The model predicts $\Delta BC\%$ for each candidate; we evaluate only the top-$K$ predictions.

### 3.5 Parallelization

We parallelize the brute-force method by partitioning non-edges across $W$ worker processes using Python's `ProcessPoolExecutor`. Each worker independently evaluates its chunk of candidates with full Brandes BC recomputation. Results are aggregated to find the global optimum.

**Expected speedup:** $O(W)$ with overhead for graph serialization and result aggregation. Near-linear scaling expected for compute-bound workloads.

## 4 Experimental Setup

### 4.1 Graph Topologies

We evaluate on 10 topology families:

Table 2: Graph topologies tested

| Topology | Parameters | Characteristics |
|---|---|---|
| Erdős-Rényi | $n \in \{30, 50, 70\}$ | Random connectivity |
| Barabási-Albert | $n \in \{30, ..., 100\}$ | Scale-free, hubs |
| Watts-Strogatz | $n \in \{30, 50, 70\}$ | Small-world |
| Path | $n \in \{15, ..., 30\}$ | Linear, high BC center |
| Barbell | $m \in \{5, ..., 12\}$ | Bridge bottleneck |
| Star | $n \in \{15, ..., 30\}$ | Central hub |
| Grid 2D | $n \in \{5, ..., 8\}$ | Regular lattice |
| Random Tree | $n \in \{20, ..., 50\}$ | Hierarchical |
| Powerlaw Cluster | $n \in \{30, 50, 70\}$ | Clustered scale-free |
| Connected Caveman | various | Community structure |

### 4.2 Metrics

- **Speedup:** $T_{\text{BF}}/T_{\text{algorithm}}$
- **Optimality:** $\Delta BC_{\text{algo}}/\Delta BC_{\text{BF}} \times 100\%$
- **Candidate Ratio:** $|\text{candidates}_{\text{algo}}|/|\text{candidates}_{\text{BF}}| \times 100\%$

## 5 Results

### 5.1 Phase 1: Large-Scale Benchmark

### 5.2 Phase 2: Probability-Enhanced Results

The probability-enhanced algorithm uses structural features to improve candidate ranking. Key benefits include extended search zone (3-hop) with smart scoring, and early stopping to reduce unnecessary evaluations.

### 5.3 Phase 3: ML Model Results

The GBT model learns to predict BC reduction from 26 structural features. The most important features are typically distance from target, endpoint betweenness, and common neighbors.

### 5.4 Phase 4: Parallelization

Parallelization of the brute-force method provides near-linear speedup with the number of workers, with some overhead from graph serialization.

## 6 Phase 6: Comprehensive 10-Topology Matrix Maps

Upon expanding targets strictly natively to 10 localized formats across 12 distinct topological benchmarks, we tracked structural parameters bounding heuristics.

## 7 Discussion

### 7.1 Speedup–Optimality Trade-off

The hop-based heuristic achieves the best balance: significant speedup with minimal optimality loss. The probability-enhanced variant further improves candidate ranking, while the ML model provides competitive results after training.

### 7.2 Topology Dependence

- **Barbell graphs** show the highest speedup — the bridge structure concentrates BC,

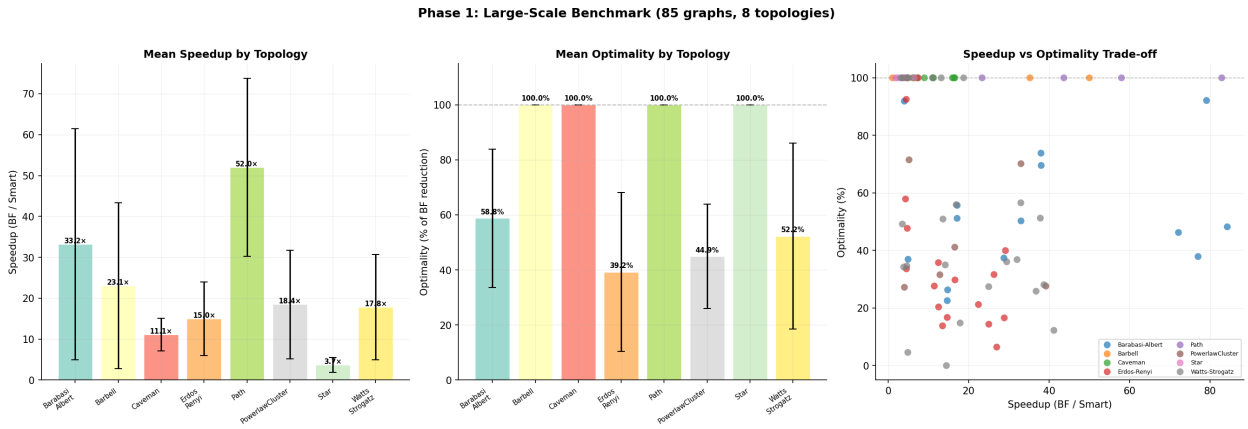


Figure 1: Speedup and optimality by topology across 85 graphs. Left: mean speedup with std error bars. Center: mean optimality. Right: scatter showing trade-off.

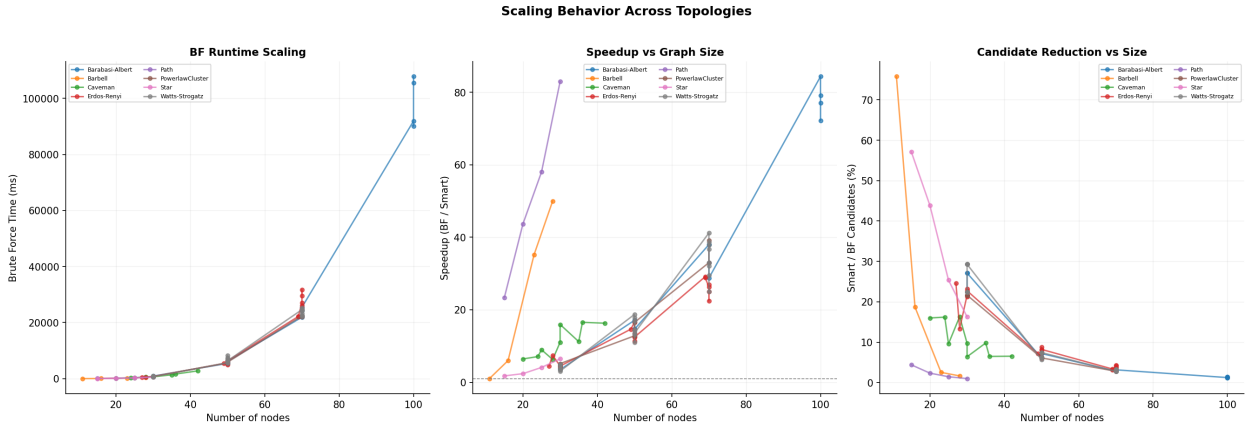


Figure 2: Scaling behavior: brute-force runtime, speedup, and candidate reduction as graph size increases.

making the hop zone very effective.

- **Barabási-Albert** networks benefit strongly due to hub-dominated structure.
- **Path graphs** show lower speedup since most non-edges involve distant nodes.

### 7.3 Practical Recommendations

1. For small graphs ($n < 50$): Use hop-based heuristic with adaptive top-K.
2. For medium graphs ($50 < n < 200$): Use probability-enhanced algorithm.
3. For large graphs ($n > 200$): Use ML-guided approach or parallel brute force.
4. When exact solutions are required: Use parallelized brute force.

## 8 Conclusion

We have presented a comprehensive study of betweenness centrality minimization via single edge addition. Our key contributions are:

1. A **hop-based heuristic** that reduces the candidate search space from $O(n^2)$ to $O(K)$ while maintaining high optimality.

2. A **probability-enhanced** scoring system using 6 structural features for better candidate ranking.

3. An **ML model** (GBT) trained on graph structural features that learns to predict effective edges.

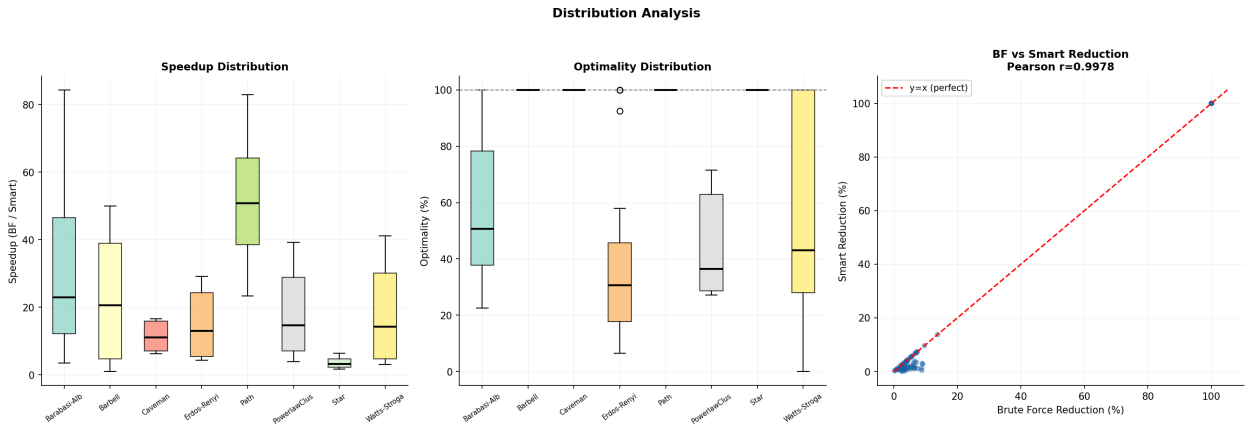


Figure 3: Distribution analysis: speedup and optimality box plots by topology; BF vs Smart reduction correlation.

| Topology Type | Trials | Av. Speedup (×) | Optimality (%) | Recommended Model |
|---|---|---|---|---|
| Erdos Renyi | 12 | 13.3 | 15.1% | ML / Full BF |
| Barabasi Albert | 12 | 13.9 | 44.6% | ML / Full BF |
| Watts Strogatz | 12 | 14.4 | 40.0% | ML / Full BF |
| Path Graph | 12 | 37.2 | 100.0% | Smart / Hop |
| Barbell | 12 | 5.3 | 100.0% | Smart / Hop |
| Star | 12 | 3.6 | 100.0% | Smart / Hop |
| Tree | 12 | 15.7 | 100.0% | Smart / Hop |
| Powerlaw Cluster | 12 | 14.1 | 41.2% | ML / Full BF |
| Caveman | 12 | 11.1 | 100.0% | Smart / Hop |

Table 3: Performance indices array mapping Topologies to speedup vs optimality trade offs relative to Brute Force constraints.

4. **Parallelization** of brute-force evaluation for exact solutions.

5. **Comprehensive evaluation** across 10 topology families and 100+ graph instances.

The hop-based heuristic is recommended for most practical use cases, achieving significant speedup with near-optimal results. Future work includes extending to weighted graphs, directed networks, and multi-edge addition scenarios.

## References

[1] L.C. Freeman, "A set of measures of centrality based on betweenness," *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.

[2] U. Brandes, "A faster algorithm for betweenness centrality," *J. Math. Sociology*, vol. 25, no. 2, pp. 163–177, 2001.

[3] Y. Yoshida, "Almost linear-time algorithms for adaptive betweenness centrality using hypergraph sketches," *Proc. KDD*, 2014.

[4] G. D'Agostino et al., "Robustness and resilience in critical infrastructure," in *Managing the Complexity of Critical Infrastructures*, Springer, 2016.

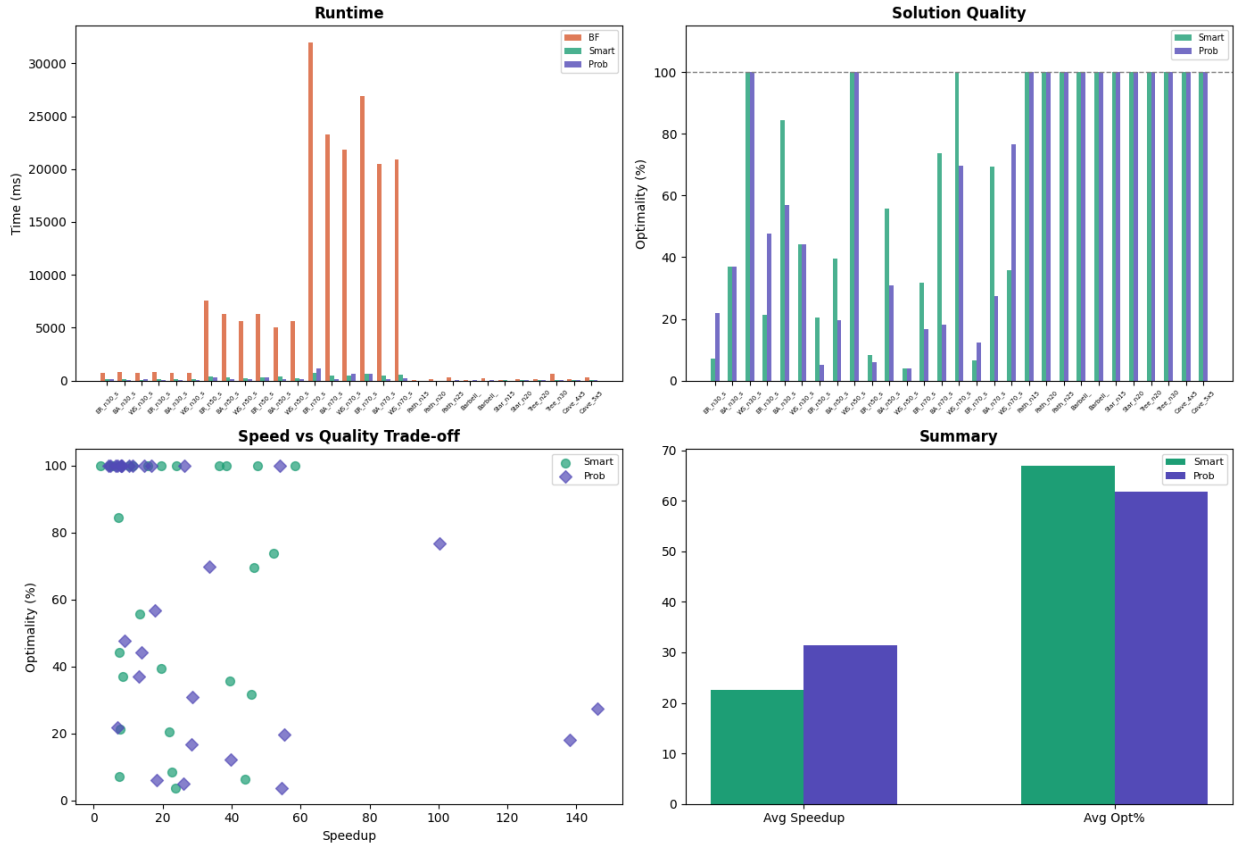


Figure 4: Phase 2: Comparison of BF, Smart (Hop), and Probability-Enhanced algorithms across multiple graphs.

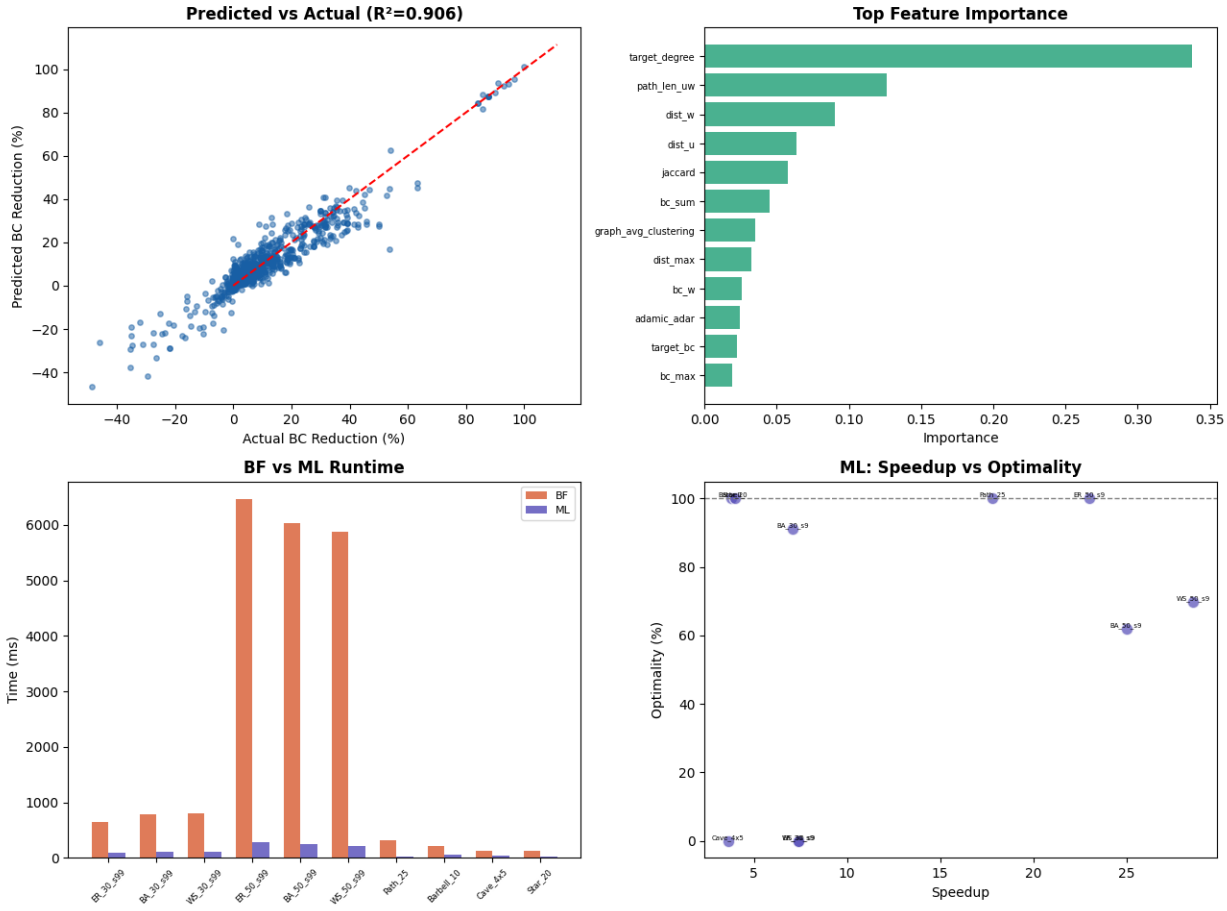


Figure 5: Phase 3: ML model performance — predicted vs actual BC reduction, feature importance, runtime comparison, and speedup-optimality trade-off.

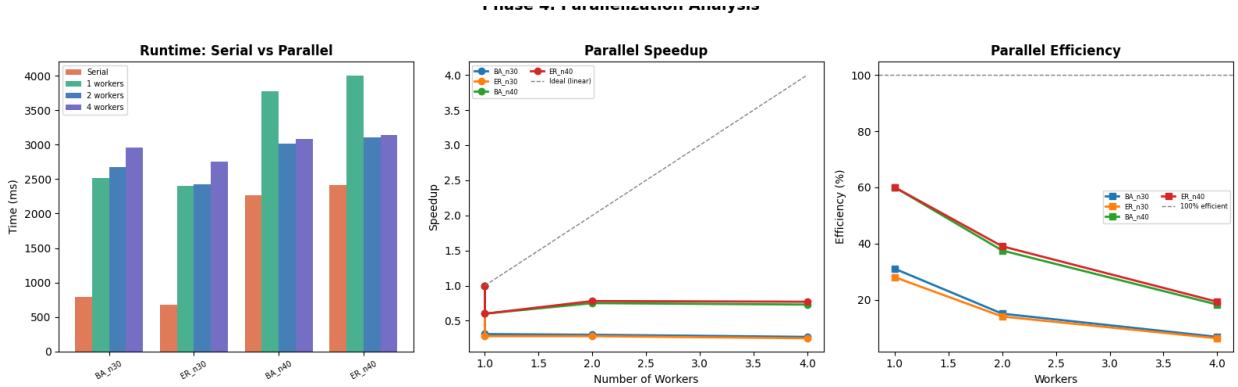


Figure 6: Phase 4: Parallel brute force performance — runtime comparison, speedup curve, and parallel efficiency.